

An Introduction to Reinforcement Learning

Caleb Lee, Catherine Cao, Arshia Mago, Ashlee King, Christina Fan^a

^a*University of Waterloo, Department of Psychology, 200 University Avenue
West, Waterloo, N2L 3G1*

Abstract

This is the abstract. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vestibulum augue turpis, dictum non malesuada a, volutpat eget velit. Nam placerat turpis purus, eu tristique ex tincidunt et. Mauris sed augue eget turpis ultrices tincidunt. Sed et mi in leo porta egestas. Aliquam non laoreet velit. Nunc quis ex vitae eros aliquet auctor nec ac libero. Duis laoreet sapien eu mi luctus, in bibendum leo molestie. Sed hendrerit diam diam, ac dapibus nisl volutpat vitae. Aliquam bibendum varius libero, eu efficitur justo rutrum at. Sed at tempus elit.

Keywords: Q-learning, Reinforcement learning, State, Agent, Action, TD learning

1. Introduction

1.1. History

- Ashlee History

1.2. Importance and Applications

- Arshia + some Catherine applications stuff

2. Mathematical Theory

- Catherine Q-Learning formula explanation

Here is an equation:

$$f_X(x) = \left(\frac{\alpha}{\beta}\right) \left(\frac{x}{\beta}\right)^{\alpha-1} e^{-\left(\frac{x}{\beta}\right)^\alpha}; \alpha, \beta, x > 0.$$

Inline equations work as well: $\sum_{i=2}^{\infty} \{\alpha_i^\beta\}$

^{*}Corresponding author

3. Methods

- Christina and Caleb ## Implementation We built and implemented a Q-learning algorithm within a maze environment. The environment was represented as a 6×6 grid constructed using a NumPy array, where integer values encoded state types and associated rewards. Specifically, cells were defined as follows: 0 represented empty traversable space, 1 represented impassable walls, 2 represented fatal states (“holes”) that returned the agent to the starting position, and 3 represented the goal state associated with a positive reward and episode termination.

A Q-table was initialized with zeros to represent the expected value of each state-action pair. The learning rate (α), future discount factor (γ), and exploration rate (ϵ) were predefined and held constant throughout training. The agent’s action space consisted of four discrete movements (up, down, left, right).

At each step, the agent selected an action using the Epsilon-Greedy strategy. State transitions were determined by applying the selected action to the agent’s current position, with resulting coordinates mapped onto the grid to determine the subsequent state and corresponding reward.

Q-values were iteratively updated using the temporal difference learning rule based on the observed reward and the maximum expected future reward from the subsequent state. Training was conducted over a fixed number of episodes, with each episode beginning at the starting state and terminating upon reaching the goal or a terminal penalty state.

3.1. Neuropsychiatry Extension

- Caleb Part
- Q learning in computational psychiatry
- Example: D1/D2 receptors

4. Results

4.1. Model

After running 1000 episodes, we present evidence of the agent’s learning process. Figure 1 illustrates changes in the number of steps the agent takes to complete one episode over the course of 500 episodes.

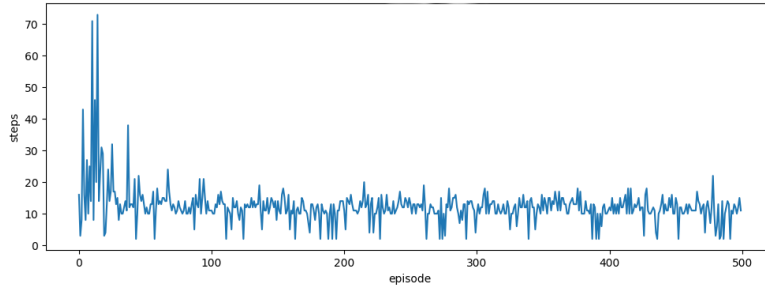


Figure 1: Changes in Steps to Complete One Episode

Figure 2 presents changes in the TD learning error (δ) over time across 6000 episodes.

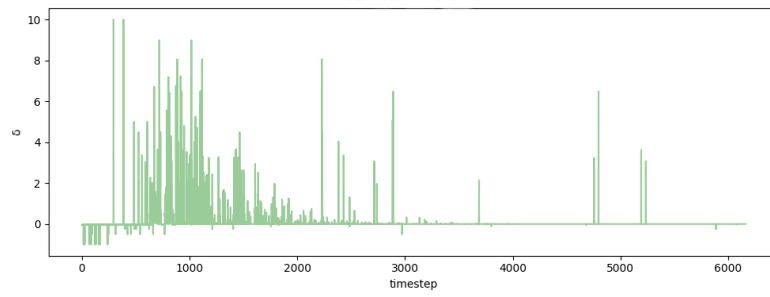


Figure 2: TD Error Over Time

Figure 3 is a Q-value heatmap showing the Q values of the best action to take in a given state (i.e. state-action pair that renders the maximal Q-value), with greater Q-values of darker color.

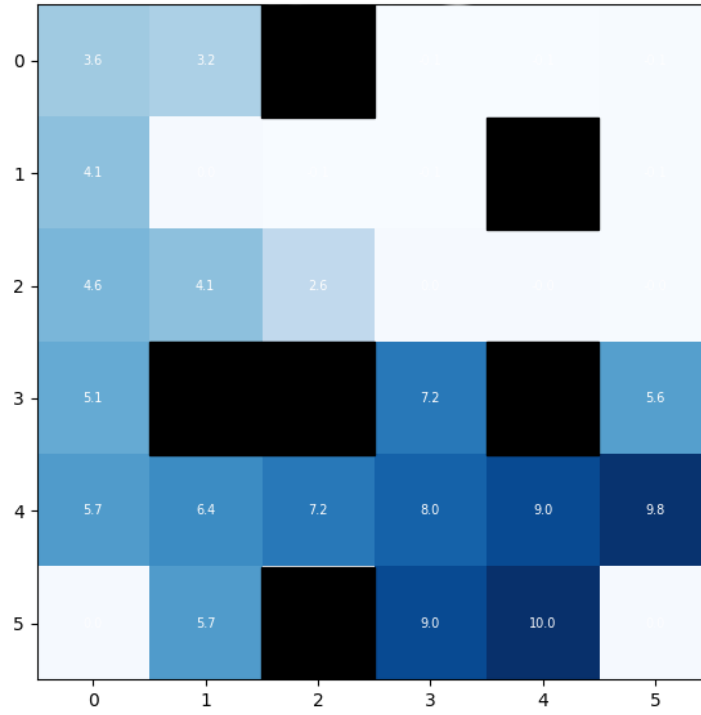


Figure 3: Q-Value Heatmap

Figure 4 illustrates the optimal actions to take in a given state, with visualizations of actions as arrows in the maze environment. This represents the optimal policy the agent is learning.

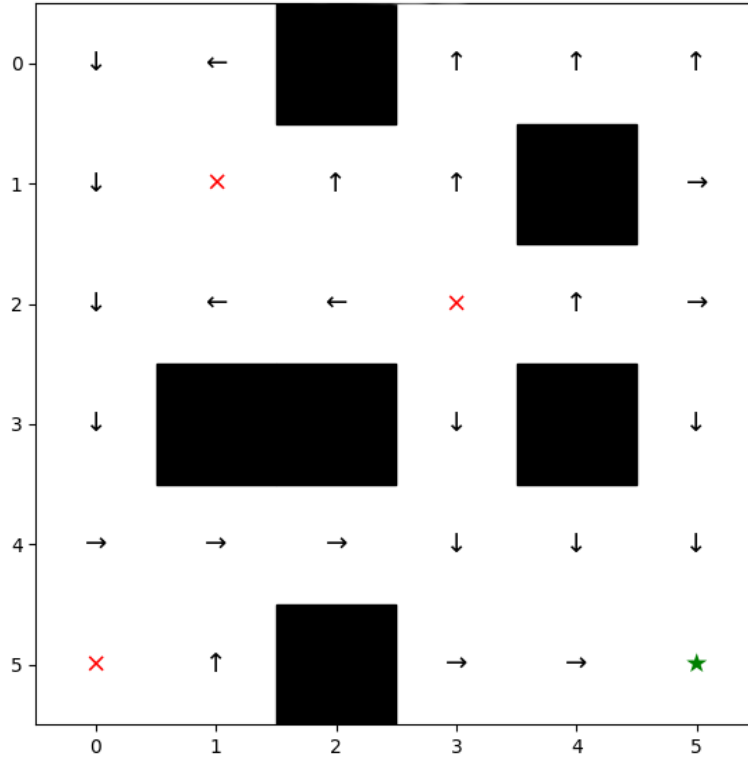


Figure 4: Optimal Learned Policy

4.2. Neuropsychiatry Extension

- Caleb

References